

Allocating Chores with Restricted Additive Costs: Achieving EFX, MMS, and Efficiency Simultaneously

Zehan Lin*
yc47490@um.edu.mo
University of Macau
Macau, China

Xiaowei Wu*
xiaoweiwu@um.edu.mo
University of Macau
Macau, China

Shengwei Zhou*
shengwei.zhou@ntu.edu.sg
Nanyang Technological University,
Singapore

Abstract

In a web-based review platform, papers from various research fields must be assigned to a group of reviewers. Each paper has an inherent cost, which represents the effort required for reading and evaluating it (e.g., the paper's length). Reviewers can bid on papers they are interested in, and if they are assigned a paper they have bid on, no cost is incurred. Otherwise, the inherent cost $c(e)$ for paper e applies. We capture this with a model of restricted additive costs: every item e has a cost $c(e)$, and each agent either incurs 0 or $c(e)$ for e . In this work, we study how to allocate such chores fairly and efficiently. We propose an algorithm for computing allocations that are both EFX and MMS. Furthermore, we show that our algorithm achieves a 2-approximation of the optimal social cost, and the approximation ratio is optimal. We also show that slightly weaker fairness guarantees can be obtained if one requires the algorithm to run in polynomial time.

CCS Concepts

• Theory of computation → Algorithmic game theory.

Keywords

Fair division, Restricted additive chores, EFX, Efficiency

ACM Reference Format:

Zehan Lin, Xiaowei Wu, and Shengwei Zhou. 2026. Allocating Chores with Restricted Additive Costs: Achieving EFX, MMS, and Efficiency Simultaneously. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792300>

1 Introduction

Fair allocation problems focus on allocating a set M of m items among a group N of n agents, where agents have heterogeneous valuation functions over the items. When the valuations are positive, the items are viewed as goods (e.g., resources), whereas negative valuations indicate that the items are chores (e.g., tasks). The objective of fair allocation is to compute an allocation that ensures fairness while ideally also achieving efficiency.

In recent years, fair division has become increasingly relevant in web-based platforms, where resources or tasks need to be allocated

among multiple agents with varying preferences. Examples include the fair allocation of advertising spaces on digital platforms, task assignments in collaborative online projects, and even the distribution of computational resources in cloud services. In such settings, ensuring fairness in the distribution of resources or responsibilities, while also optimizing social costs, presents a significant challenge. The growing importance of such scenarios in online environments motivates the study of fair allocation algorithms.

In this work, we focus on fair allocations for chores, where each agent has an additive cost function that assigns a non-negative cost to each item. Various notions have been proposed to measure the fairness of allocations. Envy-freeness (EF) is one of the most extensively studied among them. An allocation is EF if no agent prefers another agent's bundle over her own. However, EF allocations are not guaranteed to exist when items are indivisible. As a result, much attention has been given to its relaxations. Two widely studied relaxations are envy-freeness up to one item (EF1) [47] and envy-freeness up to any item (EFX) [21]. Specifically, EF1 requires that any envy between two agents can be eliminated by removing some item from the envious agent's bundle, whereas EFX requires that envy can be eliminated by removing any item from the envious agent's bundle.

EFX Allocations for Chores. The existence of EFX allocations is regarded as the central problem in the field of fair allocation and referred to as “fair allocation's most enigmatic question” by Procaccia [51]. For two agents with general cost functions, EFX allocations can be computed by the divide-and-choose algorithm. However, the existence of EFX allocations remains unknown for $n \geq 3$ agents with additive functions. A sequence of works tried to answer this question by considering important special cases. Aziz et al. [12] demonstrated that EFX allocations can be computed when the set of chores can be divided into two types, such that all agents perceive any two items of the same type as identical. For identical ordinal preference (IDO) instances, where all agents share the same preference ordering over items, Aziz et al. [10] showed that EFX allocations are guaranteed to exist. Tao et al. [55] proved the existence of EFX allocations for binary instances, where the cost of any item to any agent is either 0 or 1. The existence of EFX allocations has also been established under other structured settings, such as leveled preferences [29], lexicographic preferences [39], and when $m \leq 2n$ [34, 44]. Nevertheless, extending these results to more general settings remains highly non-trivial. Even for the seemingly modest case of bi-valued instances (which generalizes the binary instances by allowing item costs to be either 1 or some fixed constant $\epsilon \in (0, 1)$), existence results have been established only in the case of three agents or when $\epsilon = 0.5$, despite significant efforts [33, 44, 46, 56].

*The authors are ordered alphabetically.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792300>

MMS Allocations for Chores. Another well-studied fairness notion is the maximin share (MMS), introduced by Budish [19]. For each agent, the MMS is defined as the minimum cost she can guarantee for herself by partitioning all items into n bundles and then receiving the least preferred bundle. When $n = 2$, the divide-and-choose algorithm can be used to compute MMS allocations. However, for $n \geq 3$, MMS allocations are not guaranteed to exist [13, 28]. Consequently, much of the research has shifted toward computing approximate MMS allocations, e.g., see [11, 13, 14, 27, 40]. The state-of-the-art approximation ratio of MMS for chores is 13/11 by Huang and Segal-Halev [41]. On the negative side, Feige et al. [28] showed that no algorithm can achieve an approximation ratio better than 44/43, even for three agents.

Restricted Additive Functions. In the restricted additive setting¹, each item e has an inherent value $c(e)$ so that for any agent i , we have $c_i(e) \in \{0, c(e)\}$. The restricted setting was extensively studied in prior work, which mainly focus on the case of goods. The setting has been well-studied in the Santa Claus problem [8, 15, 22, 52], which focuses on the computational complexity of maximizing the minimum utility. The setting in fair allocation has also drawn significant attention recently, e.g., Akrami et al. [3] showed that EF2X² allocations for goods exist when agents have restricted additive functions. The result was partially improved by Kaviani et al. [43], who showed that EFX allocations can be computed when each item is only relevant to two agents. Mashbat et al. [17] proved the existence of MMS allocations for three agents under restricted setting. However, very few works have considered the setting for chores. The work by Sun and Chen [53] is a notable exception, who proposed a group-strategyproof randomized mechanism with best-of-both-worlds fairness guarantee. Therefore, it remains unknown whether EFX allocations exist for chores in the restricted setting. Restricted additive functions for chores can arise in many scenarios. For instance, in a web-based review platform, papers must be assigned to a group of reviewers. Each paper has an inherent cost, which represents the effort required to evaluate it (e.g., the paper's length). Reviewers can bid on papers they are interested in, and if they are assigned a paper they bid on, no cost is incurred. Otherwise, the inherent cost $c(e)$ for paper e applies.

Efficiency. Besides fairness, efficiency is another important measure of the quality of allocations. Pareto optimality (PO) is one of the most widely used measures of efficiency. An allocation is PO if no other allocation can make at least one agent strictly better off without making any other agent worse off. Another important efficiency measure is the social cost, which is the total (sum of) cost incurred by all agents. The optimal social cost can be achieved by allocating each item to the agent who incurs the least cost for it. However, this often results in a highly unfair allocation. In recent years, the existence of allocations that are both fair and efficient has drawn significant attention. Unfortunately, efficiency and fairness often conflict with each other. Requiring an allocation to satisfy certain fairness guarantees may lead to a loss in efficiency, i.e., the resulting social cost may be higher than the (unconstrained) optimal.

¹For simplicity, in the remainder of this paper we refer to the restricted additive setting as the restricted setting, when the context is clear.

²EF2X allocations for goods require that the envy between two agents can be eliminated by removing any two items from the envied agent.

To quantify the increase in social cost due to the fairness constraint, Bertsimas et al. [16] and Caragiannis et al. [20] introduced the price of fairness (PoF), which is the ratio between the optimal social cost of fair allocations and the unconstrained optimum.

1.1 Discussion on Existing Algorithms

We first discuss some natural algorithms for computing EFX allocations for chores in other related settings, and show some difficulties of extending these algorithms to the restricted setting. Via these discussions, we identify several useful observations that will inspire and guide the design of our algorithm. Observe that when $c(e) = 1$ for all $e \in M$, the restricted setting degenerates to the binary setting (in which $c_i(e) \in \{0, 1\}$ for all $i \in N$ and $e \in M$). Therefore, it is natural to consider extensions of algorithms for computing EFX allocations for binary instances to the restricted setting. In the algorithm by Tao et al. [55], the items with cost 0 to some agent are allocated first, which is natural and reasonable, as they do not incur any cost to the receiving agent. We use M^0 to denote this set of items. Then they try to allocate items with non-zero costs to all agents one by one. During this process, previously assigned items may be reallocated. Their algorithm guarantees that every item $e \in M^0$ will be allocated to an agent with zero cost on e after the reallocations. Therefore, in addition to guaranteeing EFX, the returned allocation is PO as it minimizes social cost. However, they also give a simple example showing that it is not possible to extend their algorithm to the more general restricted setting:

Agent	e_1	e_2	e_3
1	$1 + \epsilon$	1	0
2	$1 + \epsilon$	0	1

Table 1: A hard instance modified from [55], where $\epsilon > 0$.

In the above instance, if we allocate item e_2 to agent 2 and e_3 to agent 1, then no matter who receives item e_1 , the resulting allocation is not EFX. In other words, to ensure EFX, we need to allocate either e_2 to agent 1 or e_3 to agent 2. Indeed, allocating e_1 to agent 1 and the remaining items to agent 2 gives an EFX allocation.

OBSERVATION 1.1. *To compute EFX allocations, we sometimes need to allocate an item e to agent i with $c_i(e) > 0$, even if there exists another agent $j \neq i$ with $c_j(e) = 0$.*

Moreover, it can be verified that every PO allocation must allocate item e_2 to agent 2 and item e_3 to agent 1, which implies that EFX is incompatible with PO in the restricted setting.

OBSERVATION 1.2. *EFX and PO allocations may not exist in the restricted setting.*

We also consider other natural algorithms, e.g., the algorithm of Aziz et al. [10] for computing EFX allocations for IDO instances based on envy-cycle elimination. Their algorithm allocates items one by one based on a directed envy graph, where each vertex represents an agent, and a directed edge from agent i to agent j indicates that i envies j . In each round, an agent who currently does not envy any other agent is chosen to receive the unallocated

item with maximum cost, where such an agent is guaranteed to exist by eliminating directed cycles in the graph. Their algorithm guarantees EFX since every agent is EF towards all other agents after removing the last received item, and the last received item is the item with the smallest cost by the property of IDO instances. Naturally, we can extend the algorithm to the restricted setting, where the ordering of items is induced by c , e.g., we allocate items in decreasing order of $c(e)$. Unfortunately, the algorithm fails to compute EFX allocations, because if an agent i received an item e with cost 0, then even if agent i is EF towards all other agents, we cannot pick i to receive an item without violating EFX. Consider the following simple example:

Agent	e_1	e_2	e_3	e_4
1	$1 + \epsilon$	1	0	1
2	$1 + \epsilon$	0	1	1

Table 2: A hard instance for the envy-cycle elimination algorithm by Aziz et al. [10].

Following the algorithm, item e_1 will be allocated to one agent (say, agent 1) while e_2 and e_3 will be allocated to the other agent (say, agent 2). Then agent 2 will be chosen again to receive item e_4 , which results in an allocation that is not EFX, since agent 2 has received an item with cost 0. Note that allocating item e_4 to agent 1 will also violate EFX. From this example, we observe that we should be very careful in deciding the agents to receive items with cost 0, as such an operation will strengthen the fairness requirement for the agent from EFX to EF.

OBSERVATION 1.3. *If an agent i receives item e with $c_i(e) = 0$, then we need to ensure that agent i is envy-free towards others.*

In summary, given the difficulty of extending existing work to the restricted setting, the existence of EFX allocations for chores remains open. Moreover, since EFX and PO are incompatible, it is unknown whether we can have efficiency guarantees in addition to being EFX.

1.2 Our Results

In this work, we answer the above questions by proposing an algorithm for the computation of EFX allocations. In addition, we show that the returned allocation is also MMS and achieves an optimal approximation ratio for the optimal social cost.

Main Result (Theorem 3.1 and Theorem 4.1). *There exists an algorithm for computing allocations that are EFX, MMS and achieve a 2-approximation of the optimal social cost for any restricted instance. Furthermore, the approximation ratio is optimal.*

Compared to EFX, MMS is relatively easy to obtain under the restricted setting. Therefore, in the following, we mainly discuss the algorithmic ideas for the computation of EFX allocations.

Motivated by Observation 1.1, our algorithm takes a different approach from that of Tao et al. [55] and begins by allocating items $e \in M \setminus M^0$, which we refer to as M^+ . Note that for all $i \in N$ and $e \in M^+$, we have $c_i(e) = c(e)$. Therefore, we can compute an EFX

partition $\mathbf{B} = \{B_1, B_2, \dots, B_n\}$ for M^+ , and it remains to allocate items in M^0 .

A key observation is that we should not decide the assignment of bundles in $\mathbf{B}$ before the allocation of M^0 : by Observation 1.3, an agent cannot receive further zero-cost items unless she receives the best bundle in $\mathbf{B}$. Motivated by the matching-based approach [2, 18, 30, 44] for computing fair allocations, we allocate items in M^0 to bundles in $\mathbf{B}$ without deciding the assignment of bundles. Specifically, we let agents take turns to add unallocated items they consider having zero cost to their *favorite* bundle in $\mathbf{B}$. We show that the EFX-representing graph between the agents and the final bundles (where there is an edge between an agent and a bundle if the bundle is EFX-feasible to the agent) admits a perfect matching, which gives an EFX allocation. Since we may inevitably have to allocate items in M^0 to an agent with non-zero cost on the item (see Observation 1.1 and 1.2), to further guarantee a small social cost, we need to bound the increase in social cost due to these items. Fortunately, by the design of our algorithm, we can show that agents receiving these items are EF towards all other agents, and there are at most $n/2$ such agents. This allows us to establish a 2 approximation of the optimal social cost, which is optimal: for the hard instance shown in Table 1, the optimal social cost is $1 + \epsilon$ while every EFX allocation has a social cost of $2 + \epsilon$. Our result implies the price of EFX is 2 in the restricted settings. Interestingly, this stands in sharp contrast to prior work, where the price of EFX (and even EF1) has been shown to be unbounded in general instances [54].

However, our algorithm is not guaranteed to terminate in polynomial time, e.g., it is NP-hard to compute MMS allocations even for identical agents. Therefore, we also propose a polynomial-time algorithm that computes allocations that are EFX and 4/3-MMS (see Section 3.3).

1.3 Other Related Works

Due to the vast literature on fair allocation, in the following, we only review some of the most related works. For a more comprehensive review, please refer to the recent surveys by Aziz et al. [9] and Amanatidis et al. [4].

Approximate EFX Allocations. Given the difficulty of computing exact EFX allocations, researchers have focused on computing approximate EFX allocations. Zhou and Wu [56] presented a polynomial-time algorithm that computes a $(2 + \sqrt{6})$ -approximate EFX (i.e., $(2 + \sqrt{6})$ -EFX) allocation for three agents. The ratio has been improved to 2 by Afshinmehri et al. [1] and Christoforidis et al. [26]. For general additive instances, Garg et al. [34] proved the existence of 4-EFX allocations. This result was further improved to 2-EFX by Garg et al. [32]. For bi-valued instances (where the cost of an item to an agent is either 1 or ϵ), they showed that 3-EFX and PO allocations always exist. The ratio has recently been improved to $2 - \epsilon$ by Lin et al. [46].

EFX Allocations for Goods. Compared to the allocations for chores, the case of goods admits more fruitful results. The EFX allocations are shown to exist for two agents with arbitrary valuations or any number of agents with identical valuations by Plaut et al. [50]; three agents by Chaudhury et al. [24] and Akrami et al. [2]; two types of agents by Mahara [48]; allocations on graphs by Christodoulou

et al. [25]; two types of items by Gorantla et al. [35] and lexicographic preferences by Hosseini et al. [38]. Recently, Prakash et al. [42] showed that EFX allocations exist for three types of agents. For bi-valued instances, Amanatidis et al. [5] and Garg et al. [31] showed that EFX and PO allocations can be computed efficiently. Regarding the approximation of EFX allocations, Chan et al. [23] demonstrated the existence of 0.5-EFX allocations, while Amanatidis et al. [7] improved the ratio to 0.618. Recently, Amanatidis et al. [6] improved this ratio further to $2/3$ for some special cases.

2 Preliminary

We study the problem of fairly allocating a set of m indivisible items (chores), denoted by M , to a group of n agents N . We refer to a subset of items $X \subseteq M$ as a bundle. Each agent $i \in N$ is associated with an additive cost function $c_i : 2^M \rightarrow \mathbb{R}^+ \cup \{0\}$, which assigns a cost to every bundle of items. We use $\mathbf{c} = (c_1, \dots, c_n)$ to denote the cost functions of all agents. For any subset $X \subseteq M$ and item $e \in M$, we use $X + e$ to denote $X \cup \{e\}$, and $X - e$ to denote $X \setminus \{e\}$. For any $M' \subseteq M$, a partition $\mathbf{B} = (B_1, \dots, B_n)$ is a collection of disjoint subsets of M' such that $B_i \cap B_j = \emptyset$ for all $i \neq j$ and $\bigcup_{i \in N} B_i = M'$. An allocation $\mathbf{X} = (X_1, \dots, X_n)$ is an ordered partition of the item set M into n disjoint bundles, where agent i receives bundle X_i . In this paper, we distinguish the concepts of partition and allocation. A partition consists of n unordered bundles whose assignments (to the agents) have not been decided. On the other hand, in an allocation, the bundles are ordered and indexed by the agents receiving the bundles. For any integer $t \geq 1$, we use $[t]$ to denote the set $\{1, \dots, t\}$. Throughout the paper, we use $i \in N$ to denote the agents and $i \in [n]$ to index bundles in a partition. Given an instance $\mathcal{I} = (N, M, \mathbf{c})$, our goal is to compute an allocation $\mathbf{X}$ that is both fair and efficient.

In this work, we focus on instances where each agent has a restricted cost function, which is formally defined as follows.

DEFINITION 2.1 (RESTRICTED FUNCTIONS). A set $\{c_1, c_2, \dots, c_n\}$ of cost functions is restricted, if there exists a function c such that for any $i \in N$, any item $e \in M$, we have $c_i(e) \in \{0, c(e)\}$.

We refer to any instance in which all agents have such cost functions as a restricted instance³. Note that when $c(e) = 1$ for all $e \in M$, restricted instances reduce to binary instances. Next, we define the fairness notions.

DEFINITION 2.2 (EF). An allocation $\mathbf{X}$ is envy-free (EF) if for any agents $i, j \in N$, we have $c_i(X_i) \leq c_i(X_j)$.

DEFINITION 2.3 (EF1). An allocation $\mathbf{X}$ is envy-free up to one item (EF1) if for any agents $i, j \in N$, either $X_i = \emptyset$, or there exists an item $e \in X_i$ such that $c_i(X_i - e) \leq c_i(X_j)$.

DEFINITION 2.4 (EFX). An allocation $\mathbf{X}$ is envy-free up to any item (EFX) if for any agents $i, j \in N$, either $X_i = \emptyset$, or for any item $e \in X_i$, we have $c_i(X_i - e) \leq c_i(X_j)$.

DEFINITION 2.5 (MMS). Given a set of items $M' \subseteq M$, for any $i \in N$, her maximin share (MMS) value of items M' is defined as

$$\text{MMS}_i(M') := \min_{\mathbf{X} \in \Pi_n(M')} \max_{j \in N} \{c_i(X_j)\},$$

³Another natural way to extend the notion of restricted setting to chores is to assume that for every $i \in N$, $e \in M$, we have $c_i(e) \in \{\infty, c(e)\}$, where ∞ indicates that the agent cannot receive the item. However, we show that EFX allocations are not guaranteed to exist under this setting in the Appendix A.

where $\Pi_n(M')$ denotes the set of all n -partitions of M' .

For convenience, we use MMS_i to denote $\text{MMS}_i(M)$ when M is clear from the context.

DEFINITION 2.6 (α -MMS). An allocation $\mathbf{X}$ is said to satisfy the α -approximate maximin share guarantee for some $\alpha \geq 1$ if, for every agent $i \in N$,

$$c_i(X_i) \leq \alpha \cdot \text{MMS}_i.$$

Specifically, when $\alpha = 1$, the allocation is an MMS allocation.

Note that an n -partition of M is called an α -MMS partition for agent i if every bundle in the partition has cost at most $\alpha \cdot \text{MMS}_i(M)$. For the convenience of describing whether an agent is satisfied with a bundle, we define the concept of feasibility.

DEFINITION 2.7 (α -MMS-FEASIBLE). We say that a bundle $B \subseteq M$ is α -MMS-feasible to agent i if and only if $c_i(B) \leq \alpha \cdot \text{MMS}_i$.

DEFINITION 2.8 (EFX-FEASIBLE). Given a partition $\mathbf{B} = (B_1, \dots, B_n)$, a bundle B_i is EFX-feasible to agent j if and only if

$$\max_{e \in B_i} \{c_j(B_i - e)\} \leq \min_{k \in [n]} \{c_j(B_k)\}.$$

Note that whether a bundle B is α -MMS-feasible to agent i depends only on agent i and bundle B ; however, whether B is EFX-feasible also depends on the partition that contains B . For convenience, for agent i and bundle B , we further define

$$\hat{c}_i(B) = \max_{e \in B} \{c_i(B - e)\}$$

as the bundles cost after removing the item with the smallest cost.

Next, we introduce the efficiency notions.

DEFINITION 2.9 (SOCIAL COST). The social cost of an allocation $\mathbf{X}$ is defined as $sc(\mathbf{X}) = \sum_{i \in N} c_i(X_i)$.

DEFINITION 2.10 (PARETO OPTIMAL). An allocation $\mathbf{X}'$ Pareto dominates another allocation $\mathbf{X}$ if $c_i(X'_i) \leq c_i(X_i)$ for all $i \in N$ and the inequality is strict for at least one agent. An allocation $\mathbf{X}$ is Pareto optimal (PO) if $\mathbf{X}$ is not dominated by any other allocation.

3 EFX and MMS Allocations

In this section, we investigate how to compute allocations that are both EFX and MMS for restricted instances. The main result of this section is summarized as follows.

THEOREM 3.1. For restricted instances of indivisible chores, there exists an algorithm that computes allocations that satisfy both EFX and MMS to all agents.

The remainder of this section is devoted to proving the above theorem. To introduce our algorithm, we first classify the set of chores into two categories, as defined below.

DEFINITION 3.2. Chores are categorized as:

- $M^+ = \{e \in M : c(e) > 0 \text{ and } \forall i \in N, c_i(e) = c(e)\}$, which represents the set of items that have a consistent cost across all agents.
- $M^0 = M \setminus M^+$ denotes the set of remaining items, where each item incurs zero cost for at least one agent. Furthermore, for each agent $i \in N$, we use $M_i^0 = \{e \in M : c_i(e) = 0\}$ to denote the set of items that incur zero cost to agent i . We have $M^0 = \bigcup_{i \in N} M_i^0$.

For convenience, we refer to items in M_i^0 as *zero-cost items* for agent i . We note that, by definition, items in M^+ induce an instance where all agents have identical cost functions. For such instances, EFX allocations can be computed efficiently. Additionally, there may be some overlap between M_i^0 and M_j^0 for two agents i and j , meaning that an item could be a zero-cost item for multiple agents.

3.1 Description of the Algorithm

Our algorithm has three main phases (see Algorithm 1 for details).

Phase 1: Allocate items in M^+ . We first compute an MMS partition $B^+ = (B_1^+, \dots, B_n^+)$ with respect to (w.r.t.) c over items in M^+ . Then, as long as there exists any bundle that is not EFX-feasible w.r.t. c , we perform a reallocation to balance the bundles without violating the MMS-feasibility. We repeat the procedure until every bundle is EFX-feasible. We show in Lemma 3.3 that Phase 1 terminates after a finite number of steps and returns a partition in which every bundle $B' \in B^+$ is both EFX-feasible and MMS-feasible to any agent.

Phase 2: Allocate items in M^0 . In this phase, we allocate items in M^0 , and turn the partition B^+ into $B = (B_1, \dots, B_n)$. During this phase, agents take turns to add the unallocated zero-cost items to their most preferred bundle. Specifically, in agent i 's turn, the items in M_i^0 that have not been allocated are added to the bundle B_j that has the minimum $c_i(B_j)$. We say that agent i *modifies* bundle B_j . Note that it is possible that all items in M_i^0 have already been allocated. However, in this case, we still consider it a modification. Throughout Phase 2, we maintain a vector $L = (L_1, \dots, L_n)$ to keep track of the *last* agent that has modified a bundle. Specifically, we initialize $L = (0, \dots, 0)$, and update $L_j = i$ when agent i modifies B_j . Therefore, $L_j = i \in N$ indicates that agent i is the last agent that has modified bundle B_j ; $L_j = 0$ indicates that bundle B_j is not modified by any agent during Phase 2, and we have $B_j = B_j^+$.

At the end of Phase 2, we show the following:

- For any bundle $B_j \in B$ such that $L_j \neq 0$, it is MMS-feasible and EFX-feasible to agent L_j (see Lemma 3.4).
- For any bundle $B_j \in B$ with $L_j = 0$, which remains unchanged during this phase, it is MMS-feasible and EFX-feasible to all agents (see Lemma 3.5).

Phase 3: Compute the final allocation. Finally, in this phase, we decide the assignment of bundles and turn the partition B into an allocation $X = (X_1, \dots, X_n)$. For any bundle $B_j \in B$ with $L_j \neq 0$, we assign B_j to agent L_j . As we will show below, B_j is the most preferred bundle for agent L_j and therefore agent L_j is EF towards all other agents. For the remaining bundles (with $L_j = 0$), we allocate them to the remaining agents (who have not received any bundle) arbitrarily. Since these bundles only contain items from M^+ , we use Lemma 3.5 to show that the bundles are MMS-feasible and EFX-feasible to their receivers.

We illustrate the execution of our algorithm in Figure 1, where we show the modifications of agents to the bundles at the end of Phase 2. We use a blue edge between an agent and a bundle to represent a modification, where only the last agent's edge is solid.

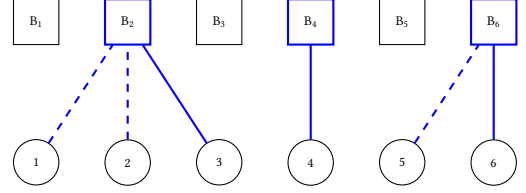


Figure 1: Illustration for the modifications by the end of Phase 2.

Algorithm 1: Computation of EFX and MMS Allocations for Restricted Instances

Input: Restricted instance $I = (N, M, c, \{c_i\}_{i \in N})$

- 1 Initialize $B_i \leftarrow \emptyset$ for all $i \in [n]$;
// Phase 1: Allocate items in M^+
- 2 Compute an MMS partition $B = (B_1, \dots, B_n)$ of M^+ w.r.t. c ;
// Modify the partition to ensure EFX⁴
- 3 **while** $\max_{B \in B} \{\hat{c}(B)\} > \min_{B \in B} \{c(B)\}$ **do**
- 4 $B_b \leftarrow \arg \max_{B \in B} \{\hat{c}(B)\}$;
- 5 $B_l \leftarrow \arg \min_{B \in B} \{c(B)\}$;
- 6 $e \leftarrow \arg \min_{e' \in B_b} \{c(e')\}$;
- 7 $B_l \leftarrow B_l + e$;
- 8 $B_b \leftarrow B_b - e$;
- 9 // Phase 2: Allocate items in M^0
- 10 $R \leftarrow M^0$; // Remaining items
- 11 **for** $i \in N$ **do**
- 12 $S_i \leftarrow M_i^0 \cap R$;
- 13 $B_j \leftarrow \arg \min_{B \in B} \{c_i(B)\}$; // Break ties arbitrary
- 14 $B_j \leftarrow B_j \cup S_i$;
- 15 $R \leftarrow R \setminus S_i$;
- 16 $L_j \leftarrow i$;
- 17 // Phase 3: Compute the final allocation
- 18 **for** $i \in [n]$ **do**
- 19 **if** $L_i \neq 0$ **then**
- 20 $X_{L_i} \leftarrow B_i$;
- 21 $N \leftarrow N \setminus \{L_i\}$;
- 22 $B \leftarrow B \setminus \{B_i\}$;
- 23 **for** $i \in N$ **do**
- 24 Pick arbitrary bundle $B \in B$;
- 25 $X_i \leftarrow B$;
- 26 $B \leftarrow B \setminus \{B\}$;

Output: EFX and MMS allocation X

3.2 Fairness Guarantee

Now we prove the fairness guarantee of the returned allocation. We first show that the partition B^+ computed in Phase 1 has strong fairness guarantees. Recall that in this phase, we only consider items in M^+ , which induce an instance with identical cost functions. Consequently, for any $i, j \in N$, we have $\text{MMS}_i(M^+) = \text{MMS}_j(M^+)$.

⁴Recall that $\hat{c}_i(B) = \max_{e \in B} \{c_i(B - e)\}$

LEMMA 3.3. *For any agent $i \in N$, every bundle $B \in \mathbf{B}^+$ is MMS-feasible and EFX-feasible.*

PROOF. Recall that we start with an MMS partition of the items M^+ , and perform reallocations until all bundles in the partition are EFX-feasible. Therefore, it suffices to show that the algorithm terminates after a finite number of steps, and the MMS-feasibility of the bundles is maintained throughout the reallocations.

In the following, we fix an arbitrary agent $i \in N$. By the definition of MMS partition, at the beginning of Phase 1, for any bundle $B \in \mathbf{B}^+$ we have

$$c_i(B) = c(B) \leq \text{MMS}_i(M^+) \leq \text{MMS}_i(M),$$

where the second inequality follows from the fact that $M^+ \subseteq M$. Therefore, to show that all bundles are MMS-feasible to agent i at the end of Phase 1, it suffices to show that the reallocations do not increase $\max_{j \in [n]} \{c_i(B_j)\}$.

Consider any reallocation, in which item e is reallocated from bundle B_b to B_l . By the condition of the while-loop, we have $c_i(B_b - e) > c_i(B_l)$. Thus, we have:

$$c_i(B_l + e) = c(B_l + e) < c(B_b) = c_i(B_b).$$

Therefore, after the reallocation, the cost of the two new bundles is upper bounded by $c_i(B_b)$, which implies that $\max_{j \in [n]} \{c_i(B_j)\}$ does not increase after the reallocation.

It remains to prove that the while-loops terminate after a finite number of rounds. We show this by introducing a potential function $\sum_{j \in [n]} (c(B_j))^2$. Clearly, the potential function is upper bounded by $(c(M^+))^2$ and lower bounded by 0. Each reallocation of item e from B_b to B_l decreases this potential by

$$\begin{aligned} & (\hat{c}(B_b) + c(e))^2 + (c(B_l))^2 - (\hat{c}(B_b))^2 - (c(B_l) + c(e))^2 \\ & \geq 2 \cdot c(e) \cdot (\hat{c}(B_b) - c(B_l)) > 0. \end{aligned}$$

Therefore, the potential strictly decreases, implying that Phase 1 will terminate after a finite number of reallocations. $\square$

Remark. Note that we can obtain the same fairness guarantee (for EFX and MMS) if we replace $\mathbf{B}^+$ by a Leximin allocation of M^{+5} . However, showing that the Leximin allocation ensures EFX and MMS requires a proof that is almost identical to what we presented above. Furthermore, while our algorithm is not guaranteed to terminate in polynomial time, it is constructive, which is slightly stronger than the existential guarantee given by Leximin allocations.

Next, we analyze Phase 2. In the following two lemmas, we show that if a bundle is modified during this phase, it will be MMS-feasible and EFX-feasible to the agent who performs the last modification, at the end of Phase 2; if it is not modified by any agent, then it is MMS-feasible and EFX-feasible to all agents.

LEMMA 3.4. *For any bundle $B_j \in \mathbf{B}$ with $L_j = i \neq 0$, B_j is MMS-feasible and EFX-feasible to agent i .*

PROOF. The result follows from the design of Phase 2. In Phase 2, each agent modifies her favorite bundle in turn. By definition, L_j is the last agent who has modified bundle B_j . Therefore, bundle B_j is agent i 's favorite bundle before the modification takes place.

⁵Here, Leximin refers to the Leximin++ allocation introduced by Plaut and Roughgarden [50].

Furthermore, the modification does not increase the cost of B_j w.r.t. c_i . From that moment on, the costs of all other bundles (w.r.t. c_i) do not decrease during this phase. Consequently, B_j remains agent i 's favorite bundle at the end of Phase 2. Therefore, we have

$$c_i(B_j) \leq \max_{k \in [n]} \{c_i(B_k^+)\} \leq \text{MMS}_i(M^+) \leq \text{MMS}_i,$$

where the first inequality holds since there is at least one bundle that is not modified before agent i performs the modification. $\square$

LEMMA 3.5. *For bundle $B_j \in \mathbf{B}$ with $L_j = 0$, B_j is MMS-feasible and EFX-feasible to every agent $i \in N$.*

PROOF. By Lemma 3.3, every bundle $B_j \in \mathbf{B}$ with $L_j = 0$ is MMS-feasible and EFX-feasible to every agent $i \in N$ at the beginning of Phase 2. Since $L_j = 0$, bundle B_j is not modified during Phase 2. Since the cost of each bundle does not decrease during Phase 2, B_j continues to satisfy the fairness guarantee for every agent $i \in N$ at the end of Phase 2. $\square$

Finally, we analyze the allocation $\mathbf{X}$ computed in Phase 3. Given Lemma 3.4 and Lemma 3.5, we know that in the final allocation, for all $i \in N$, the bundle X_i assigned to agent i is MMS-feasible and EFX-feasible to agent i . Therefore, the final computed allocation is MMS and EFX.

3.3 Polynomial-Time Algorithm

In this section, we present a polynomial-time algorithm that computes EFX and $\frac{4}{3}$ -MMS allocations (See Algorithm 2).

Note that the main bottleneck in the time complexity of Algorithm 1 lies in Phase 1⁶. To overcome this, we introduce an alternative method to compute the initial partition in polynomial time, allowing us to replace Phase 1 while preserving the correctness and structure of Phase 2 and Phase 3. In particular, Barman and Krishnamurthy [14] and Aziz et al. [10] proved that for IDO instances, the envy-cycle elimination algorithm yields an allocation that is $\frac{4}{3}$ -MMS and EFX, respectively. Given that agents have identical valuations in Phase 1, their algorithm becomes equivalent to our Phase 1 procedure, proceeding without any cycle rotation.

Algorithm 2: Computation of EFX and $\frac{4}{3}$ -MMS Allocations for Restricted Instances

Input: Restricted instance $I = (N, M, c, \{c_i\}_{i \in N})$

- 1 $m \leftarrow |M^+|$;
- 2 Initialize $B_i \leftarrow \emptyset$ for all $i \in [n]$;
- 3 Sort items in M^+ in decreasing order w.r.t. c : $\{e_1, \dots, e_m\}$;
- 4 **for** $j \in [m]$ **do**
- 5 $B_l \leftarrow \arg \min_{B \in \mathbf{B}} \{c(B)\}$;
- 6 $B_l \leftarrow B_l + e_j$;
- 7 Run Phases 2 and 3 on $\mathbf{B} = (B_1, \dots, B_n)$ to compute allocation $\mathbf{X}$;

Output: EFX and $\frac{4}{3}$ -MMS allocation $\mathbf{X}$

In the following, we prove the fairness guarantee for the partition $\mathbf{B}$ computed in Algorithm 2.

⁶For identical valuations, the MMS partition always exists. However, computing such a partition is NP-hard.

LEMMA 3.6. *For any agent $i \in N$, every bundle $B \in \mathbf{B}$ is EFX-feasible and $c_i(B) \leq \frac{4}{3} \cdot \text{MMS}_i(M^+)$.*

PROOF. We first show that each bundle $B \in \mathbf{B}$ is EFX-feasible with respect to all agents. In each iteration of Algorithm 2, we select the bundle B_l with the least total cost and assign to it the largest remaining item e (since items are sorted in non-increasing order of cost). Let B be any other bundle such that $B \neq B_l$. Then we have:

$$\hat{c}(B_l + e) = c(B_l) \leq c(B),$$

where the equality holds because $c(e) \leq c(e')$ for all $e' \in B_l$, and the inequality follows from the choice of B_l as the minimum-cost bundle in that iteration. This implies that bundle B_l remains EFX-feasible after adding item e . Moreover, for any other bundle $B \neq B_l$, items in B remain unchanged during this step. Thus, the EFX-feasibility of B is preserved. Therefore, all bundles remain EFX-feasible throughout the execution of the algorithm.

Next, we show that $c(B) \leq \frac{4}{3} \cdot \text{MMS}_i(M^+)$. In fact, our algorithm can be naturally interpreted as an instance of the classical job scheduling problem, where m items correspond to jobs and n bundles correspond to identical machines. In this setting, our approach is equivalent to applying the Longest Processing Time First (LPT) algorithm, which guarantees a makespan no greater than $\left(\frac{4}{3} - \frac{1}{3n}\right) \cdot \text{OPT}$ [36]. Therefore, each bundle incurs a cost of at most $\frac{4}{3} \cdot \text{MMS}_i(M^+)$. $\square$

We note that the partition computed in Phase 1 can be replaced by $\mathbf{B}$, after which the algorithm proceeds directly to Phases 2 and 3. The fairness guarantees are preserved throughout Phases 2 and 3, following similar arguments to those presented in Section 3.2. Therefore, the final allocation is EFX and $\frac{4}{3}$ -MMS.

We now analyze the overall time complexity of Algorithm 2. The construction of the initial EFX and $\frac{4}{3}$ -MMS partition involves exactly m iterations. In each iteration, identifying the least costly bundle takes $O(n)$ time, resulting in a total time complexity of $O(nm)$. In Phase 2, the for-loop executes exactly n iterations. During each iteration, the item set S_j and the bundle B_j can be identified in $O(m)$ and $O(n)$ time, respectively. The updates to B_j and R require $O(m)$ time. In Phase 3, the final allocation can be computed in $O(n)$. Therefore, the entire algorithm runs in polynomial time. Thus, we obtain the following theorem.

THEOREM 3.7. *For restricted instances of indivisible chores, there exists an algorithm that computes allocations that satisfy both EFX and $\frac{4}{3}$ -MMS to all agents in polynomial time.*

Motivated by the incompatibility between EFX and PO, we also study how to maintain fairness guarantees while ensuring PO. For general instances, Mahara [49] established the existence of allocations that satisfy both EF1 and PO. We show that EF1 and PO allocations can be achieved while simultaneously preserving the MMS guarantee for restricted instances.

THEOREM 3.8. *For restricted instances of indivisible chores, there exists an algorithm that computes allocations that satisfy both EF1, MMS and PO to all agents.*

Note that computing an MMS partition for an agent is NP-hard. However, by using a PTAS from [37], we can directly obtain the following corollary.

COROLLARY 3.9. *For restricted instances of indivisible chores, there exists an algorithm that computes allocations that satisfy both EF1, $(1 - \epsilon)$ -MMS⁷ and PO to all agents in polynomial time.*

Due to the limited space, we defer the proof of Theorem 3.8 and Corollary 3.9 in Appendix B.

4 Efficiency Guarantee

In this section, we investigate the efficiency guarantee of Algorithm 1. Since EFX and PO allocations may not exist for restricted instances, we evaluate the social cost of the allocation returned by our algorithm. Note that the optimal social cost can be achieved by allocating each item to the agent who incurs the least cost for it, for a restricted instance $\mathcal{I}$, the optimal social cost (denoted by $\text{OPT}(\mathcal{I})$) is equal to $c(M^+)$.

In the following, we show that our algorithm achieves a 2-approximation for the optimal social cost. Furthermore, we show that this approximation factor is optimal among algorithms guaranteeing EFX⁸.

THEOREM 4.1. *Algorithm 1 achieves a 2-approximation of the optimal social cost for any restricted instance. Furthermore, the approximation ratio is optimal.*

We first recall the instance shown in Table 1. It is easy to see that the optimal social cost is $1 + \epsilon$, while any EFX allocation has a social cost of $2 + \epsilon$. Hence, no EFX allocation can achieve an approximation ratio of optimal social cost better than 2.

Next, we show that the allocation returned by Algorithm 1 achieves an approximation ratio of 2 to the optimal social cost, and thus is optimal. Let the final allocation be $\mathbf{X} = (X_1, \dots, X_n)$, where X_i is assigned to agent i . Recall that during Phase 2, there could be multiple agents who have modified bundle X_i . Therefore, it is possible that $c_i(X_i)$ is strictly larger than $c_i(X_i \cap M^+)$: this happens when some other agent j modified X_i before agent i , during which some items from $M_j^0 \setminus M_i^0$ are added to X_i . Indeed, this is why we can only guarantee an approximation of the optimal social cost, rather than achieving $sc(\mathbf{X}) = c(M^+)$.

For any bundle $X \in \mathbf{X}$, we define $d(X)$ as the number of agents who modify X during Phase 2. Based on the values of $d(X_i)$, we partition the agents into three groups:

- $N_0 = \{i \in N : d(X_i) = 0\}$;
- $N_1 = \{i \in N : d(X_i) = 1\}$;
- $N_2 = \{i \in N : d(X_i) \geq 2\}$.

Consider the instance illustrated in Figure 1. By the design of the algorithm, bundles B_2 , B_4 , and B_6 are assigned to agents 3, 4, and 6, respectively. Suppose the remaining bundles B_1 , B_3 , and B_5 are assigned to agents 1, 2, and 5, respectively. Then we have $N_0 = \{1, 2, 5\}$, $N_1 = \{4\}$, and $N_2 = \{3, 6\}$.

Note that for all $i \in N_0$, we have $X_i \subseteq M^+$ since it is not modified during Phase 2; for all $i \in N_1$, we have $X_i \subseteq M^+ \cup M_i^0$ since agent i is the only agent who has modified X_i . For these agents we have

⁷ $(1 - \epsilon)$ -MMS allocations guarantee that every agent i receives a bundle whose cost is at most $(1 - \epsilon) \cdot \text{MMS}_i$, where $\epsilon > 0$.

⁸ The analysis of the efficiency guarantee for Algorithm 1 and Algorithm 2 is similar, and both achieve 2 approximation ratio for the optimal social cost. Therefore, we focus on analyzing the efficiency of Algorithm 1 in the remainder of this section.

$c_i(X_i) = c_i(X_i \cap M^+)$. Therefore, the increase in social cost is due to agents in N_2 .

Next, we introduce a lemma that upper bounds the number of agents in N_2 , which will be useful in bounding the total cost.

LEMMA 4.2. *We have $|N_0| \geq |N_2|$.*

PROOF. Since each of the n agents performs exactly one modification in Phase 2, the total number of modifications across all bundles in Phase 2 is

$$\sum_{i \in N_1} d(X_i) + \sum_{i \in N_2} d(X_i) = n.$$

Since $d(X_i) = 1$ for any agent $i \in N_1$ and $d(X_i) \geq 2$ for any agent $i \in N_2$, we have:

$$n = \sum_{i \in N_1} d(X_i) + \sum_{i \in N_2} d(X_i) \geq |N_1| + 2|N_2|,$$

which implies $|N_0| \geq |N_2|$ since $|N_0| + |N_1| + |N_2| = n$. $\square$

Next, we provide upper bounds for the total costs incurred by agents from each group.

LEMMA 4.3. *We have the following properties:*

- $\sum_{i \in N_0} c_i(X_i) + \sum_{i \in N_1} c_i(X_i) \leq c(M^+)$;
- $\sum_{i \in N_2} c_i(X_i) \leq c(M^+)$.

PROOF. As argued above, for all $i \in N_0 \cup N_1$ we have $c_i(X_i) = c_i(X_i \cap M^+) = c(X_i \cap M^+)$, since X_i is either not modified, or only modified by agent i . Therefore, we can bound the social cost incurred by agents in N_0 and N_1 by

$$\sum_{i \in N_0} c_i(X_i) + \sum_{i \in N_1} c_i(X_i) = \sum_{i \in N_0 \cup N_1} c(X_i \cap M^+) \leq c(M^+).$$

Next, we show that $\sum_{i \in N_2} c_i(X_i) \leq c(M^+)$. Fix any agent $i \in N_2$. By the design of the algorithm, X_i is the most preferred bundle for i in the final allocation, which implies that its cost is no greater than that of any bundle in $\mathbf{X}$. In particular, we have $c_i(X_i) \leq c_i(X_j)$ for all $j \in N_0$. Let $k \in N_2$ be the agent with the maximum cost $c_k(X_k)$. Combined with $|N_0| \geq |N_2|$ by Lemma 4.2, we have

$$\begin{aligned} \sum_{i \in N_2} c_i(X_i) &\leq |N_2| \cdot c_k(X_k) \leq |N_0| \cdot c_k(X_k) \\ &\leq \sum_{j \in N_0} c_k(X_j) = \sum_{j \in N_0} c(X_j) \leq c(M^+). \end{aligned}$$

where the equality holds since for all $j \in N_0$, X_j contains only items from M^+ . $\square$

Lemma 4.3 immediately implies Theorem 4.1 since

$$sc(\mathbf{X}) = \sum_{i \in N} c_i(X_i) = \sum_{i \in N_0 \cup N_1} c_i(X_i) + \sum_{i \in N_2} c_i(X_i) \leq 2 \cdot c(M^+).$$

Interestingly, our algorithm and analysis imply a tight characterization for the price of fairness (PoF) for EFX.

DEFINITION 4.4 (PoF FOR EFX). *The price of fairness for EFX on instance $\mathcal{I}$ is defined as*

$$PoF(\mathcal{I}) = \min_{\text{EFX allocation } \mathbf{X}} \left\{ \frac{sc(\mathbf{X})}{OPT(\mathcal{I})} \right\},$$

where $OPT(\mathcal{I})$ denotes the minimum social cost over all allocations, without any fairness constraint. Moreover, the price of fairness for EFX, denoted by PoF , is defined as the worst-case ratio over all instances, i.e., $PoF = \sup_{\mathcal{I}} \{PoF(\mathcal{I})\}$.

Therefore, we can get the following corollary directly.

COROLLARY 4.5. *For restricted instances, the price of EFX is 2.*

Furthermore, we also consider the cost of fairness (CoF) [45], which is defined as the worst-case additive difference between the optimal social cost under fairness constraints and the optimal social cost without such constraints, assuming that $c_i(M) \leq 1$ for all $i \in N$. Equivalently, we define it as follows.

DEFINITION 4.6 (CoF FOR EFX). *The cost of fairness for EFX is defined as*

$$\sup_{\mathcal{I}} \min_{\text{EFX allocation } \mathbf{X}} \left\{ \frac{sc(\mathbf{X}) - OPT(\mathcal{I})}{\max_{i \in N} c_i(M)} \right\}.$$

A hard instance, adapted from Table 1, establishes a lower bound of $1/2$ for the cost of EFX (see Table 3). It is easy to see that the optimal social cost is $1/2 + \epsilon$, while any EFX allocation has a social cost of 1. Hence, the cost of EFX is at least $1 - (1/2 + \epsilon) \rightarrow 1/2$ when $\epsilon \rightarrow 0$.

Agent	e_1	e_2	e_3
1	$1/2 + \epsilon$	$1/2 - \epsilon$	0
2	$1/2 + \epsilon$	0	$1/2 - \epsilon$

Table 3: Hard instance for the cost of EFX
(ϵ is a sufficiently small number)

It suffices to show that the upper bound is $1/2$. Due to space limitations, we defer the proof to Appendix C.

Thus, we obtain the following corollary.

COROLLARY 4.7. *For restricted instances, the cost of EFX is $1/2$.*

5 Conclusion and Open Problems

In this paper, we study the computation of fair and efficient allocations for indivisible chores in the restricted setting. We propose an algorithm for computing allocations that are EFX, MMS, and achieve an optimal 2-approximation of the optimal social cost. An important open problem is whether EFX allocations (even without any efficiency guarantee) exist for other classes of instances, e.g., bi-valued setting. For the allocation of goods, the existence of EFX allocations for restricted instances also remains open.

Acknowledgments

Xiaowei Wu is funded by the Science and Technology Development Fund (FDCT), Macau SAR (file no. 0147/2024/RIA2, 0014/2022/AFJ, 0085/2022/A, and 001/2024/SKL). This work is partially supported by the Ministry of Education, Singapore, under its Academic Research Fund Tier 1 (RG98/23).

References

- [1] Mahyar Afshinmehr, Matin Ansari pour, Alireza Danaei, and Kurt Mehlhorn. 2024. Approximate EFX and Exact tEFX Allocations for Indivisible Chores: Improved Algorithms. *CoRR* abs/2410.18655 (2024).
- [2] Hannaneh Akrami, Noga Alon, Bhaskar Ray Chaudhury, Jugal Garg, Kurt Mehlhorn, and Ruta Mehta. 2025. EFX: a simpler approach and an (almost) optimal guarantee via rainbow cycle number. *Operations Research* 73, 2 (2025), 738–751.
- [3] Hannaneh Akrami, Rojin Rezvan, and Masoud Seddighin. 2022. An EFX Allocation Protocol for Restricted Additive Valuations. In *IJCAI*. ijcai.org, 17–23.
- [4] Georgios Amanatidis, Haris Aziz, Georgios Birmas, Aris Filos-Ratsikas, Bo Li, Hervé Moulin, Alexandros A. Voudouris, and Xiaowei Wu. 2023. Fair division of indivisible goods: Recent progress and open questions. *Artif. Intell.* 322 (2023), 103965.
- [5] Georgios Amanatidis, Georgios Birmas, Aris Filos-Ratsikas, Alexandros Hollender, and Alexandros A. Voudouris. 2021. Maximum Nash welfare and other stories about EFX. *Theor. Comput. Sci.* 863 (2021), 69–85.
- [6] Georgios Amanatidis, Aris Filos-Ratsikas, and Alkmini Sgouritsa. 2024. Pushing the Frontier on Approximate EFX Allocations. In *EC*. ACM, 1268–1286.
- [7] Georgios Amanatidis, Evangelos Markakis, and Apostolos Ntokos. 2020. Multiple birds with one stone: Beating 1/2 for EFX and GMMS via envy cycle elimination. *Theor. Comput. Sci.* 841 (2020), 94–109.
- [8] Arash Asadpour, Uriel Feige, and Amin Saberi. 2012. Santa claus meets hypergraph matchings. *ACM Trans. Algorithms* 8, 3 (2012), 24:1–24:9.
- [9] Haris Aziz, Bo Li, Hervé Moulin, and Xiaowei Wu. 2022. Algorithmic fair allocation of indivisible items: a survey and new questions. *SIJGecom Exch.* 20, 1 (2022), 24–40.
- [10] Haris Aziz, Bo Li, Hervé Moulin, Xiaowei Wu, and Xinran Zhu. 2024. Almost proportional allocations of indivisible chores: Computation, approximation and efficiency. *Artif. Intell.* 331 (2024), 104118. doi:10.1016/J.ARTINT.2024.104118
- [11] Haris Aziz, Bo Li, and Xiaowei Wu. 2024. Approximate and strategyproof maximin share allocation of chores with ordinal preferences. *Math. Program.* 203, 1 (2024), 319–345.
- [12] Haris Aziz, Jeremy Lindsay, Angus Ritossa, and Mashbat Suzuki. 2023. Fair Allocation of Two Types of Chores. In *AAMAS*. ACM, 143–151.
- [13] Haris Aziz, Gerhard Rauchecker, Guido Schryen, and Toby Walsh. 2017. Algorithms for Max-Min Share Fair Allocation of Indivisible Chores. In *AAAI*. AAAI Press, 335–341.
- [14] Siddharth Barman and Sanath Kumar Krishnamurthy. 2020. Approximation Algorithms for Maximin Fair Division. *ACM Trans. Economics and Comput.* 8, 1 (2020), 5:1–5:28.
- [15] MohammadHossein Bateni, Moses Charikar, and Venkatesan Guruswami. 2009. MaxMin allocation via degree lower-bounded arborescences. In *STOC*. ACM, 543–552.
- [16] Dimitris Bertsimas, Vivek F. Farias, and Nikolaos Trichakis. 2011. The Price of Fairness. *Oper. Res.* 59, 1 (2011), 17–31.
- [17] Sirin Botan, Angus Ritossa, Mashbat Suzuki, and Toby Walsh. 2023. Maximin Fair Allocation of Indivisible Items Under Cost Utilities. In *SAGT (Lecture Notes in Computer Science, Vol. 14238)*. Springer, 221–238.
- [18] Xiaolin Bu, Zihao Li, Shengxin Liu, Jiaxin Song, and Biaoshuai Tao. 2024. Fair division of indivisible goods with comparison-based queries. In *WINE*.
- [19] Eric Budish. 2011. The combinatorial assignment problem: Approximate competitive equilibrium from equal incomes. *Journal of Political Economy* 119, 6 (2011), 1061–1103.
- [20] Ioannis Caragiannis, Christos Kaklamanis, Panagiotis Kanellopoulos, and Maria Kyropoulou. 2012. The Efficiency of Fair Division. *Theory Comput. Syst.* 50, 4 (2012), 589–610.
- [21] Ioannis Caragiannis, David Kurokawa, Hervé Moulin, Ariel D. Procaccia, Nisarg Shah, and Junxing Wang. 2019. The Unreasonable Fairness of Maximum Nash Welfare. *ACM Trans. Economics and Comput.* 7, 3 (2019), 12:1–12:32.
- [22] Deeparnab Chakrabarty, Julia Chuzhoy, and Sanjeev Khanna. 2009. On Allocating Goods to Maximize Fairness. In *FOCS*. IEEE Computer Society, 107–116.
- [23] Hau Chan, Jing Chen, Bo Li, and Xiaowei Wu. 2019. Maximin-Aware Allocations of Indivisible Goods. In *IJCAI*. ijcai.org, 137–143.
- [24] Bhaskar Ray Chaudhury, Jugal Garg, and Kurt Mehlhorn. 2024. EFX Exists for Three Agents. *J. ACM* 71, 1 (2024), 4:1–4:27.
- [25] George Christodoulou, Amos Fiat, Elias Koutsoupias, and Alkmini Sgouritsa. 2023. Fair allocation in graphs. In *EC*. ACM, 473–488.
- [26] Vasilis Christoforidis and Christodoulos Santorinaios. 2024. On the Pursuit of EFX for Chores: Non-existence and Approximations. In *IJCAI*. ijcai.org, 2713–2721.
- [27] Uriel Feige and Xin Huang. 2023. On picking sequences for chores. In *EC*. ACM, 626–655.
- [28] Uriel Feige, Ariel Sapir, and Laliv Tauber. 2021. A Tight Negative Example for MMS Fair Allocations. In *WINE (Lecture Notes in Computer Science, Vol. 13112)*. Springer, 355–372.
- [29] Yotam Gafni, Xin Huang, Ron Lavi, and Inbal Talgam-Cohen. 2023. Unified Fair Allocation of Goods and Chores via Copies. *ACM Trans. Economics and Comput.* 11, 3-4 (2023), 1–27.
- [30] Jugal Garg, Pooja Kulkarni, and Rucha Kulkarni. 2023. Approximating Nash social welfare under submodular valuations through (un) matchings. *ACM Transactions on Algorithms* 19, 4 (2023), 1–25.
- [31] Jugal Garg and Aniket Murhekar. 2023. Computing fair and efficient allocations with few utility values. *Theor. Comput. Sci.* 962 (2023), 113932. doi:10.1016/J.TCS.2023.113932
- [32] Jugal Garg and Aniket Murhekar. 2026. Existence of 2-EFX Allocations of Chores. In *AAAI*. AAAI Press.
- [33] Jugal Garg, Aniket Murhekar, and John Qin. 2023. New Algorithms for the Fair and Efficient Allocation of Indivisible Chores. In *IJCAI*. ijcai.org, 2710–2718.
- [34] Jugal Garg, Aniket Murhekar, and John Qin. 2025. Constant-Factor EFX Exists for Chores. In *STOC*. ACM, 1580–1589.
- [35] Pranay Gorantla, Kunal Marwaha, and Santhoshini Velusamy. 2023. Fair allocation of a multiset of indivisible items. In *SODA*. SIAM, 304–331.
- [36] Ronald L. Graham. 1966. Bounds for certain multiprocessing anomalies. *Bell system technical journal* 45, 9 (1966), 1563–1581.
- [37] Dorit S. Hochbaum and David B. Shmoys. 1987. Using dual approximation algorithms for scheduling problems theoretical and practical results. *J. ACM* 34, 1 (1987), 144–162.
- [38] Hadi Hosseini, Sujoy Sikdar, Rohit Vaish, and Lirong Xia. 2021. Fair and Efficient Allocations under Lexicographic Preferences. In *AAAI*. AAAI Press, 5472–5480.
- [39] Hadi Hosseini, Sujoy Sikdar, Rohit Vaish, and Lirong Xia. 2023. Fairly Dividing Mixtures of Goods and Chores under Lexicographic Preferences. In *AAMAS*. ACM, 152–160.
- [40] Xin Huang and Pinyan Lu. 2021. An Algorithmic Framework for Approximating Maximin Share Allocation of Chores. In *EC*. ACM, 630–631.
- [41] Xin Huang and Erel Segal-Halevi. 2023. A Reduction from Chores Allocation to Job Scheduling. In *EC*. ACM, 908.
- [42] Vishwa Prakash HV, Pratik Ghosal, Prajakta Nimbhorkar, and Nithin Varma. 2025. EFX Exists for Three Types of Agents. In *EC*. ACM, 101–128.
- [43] Alireza Kaviani, Masoud Seddighin, and AmirMohammad Shahrezaei. 2024. Almost envy-free allocation of indivisible goods: A tale of two valuations. In *International Conference on Web and Internet Economics*. Springer, 261–276.
- [44] Yusuke Kobayashi, Ryoga Mahara, and Souta Sakamoto. 2025. EFX allocations for indivisible chores: Matching-based approach. *Theoretical Computer Science* 1026 (2025), 115010.
- [45] Bo Li, Ankang Sun, and Shiji Xing. 2024. A Fair Allocation is Approximately Optimal for Indivisible Chores, or Is It? In *International Conference on Web and Internet Economics*. Springer, 564–590.
- [46] Zehan Lin, Xiaowei Wu, and Shengwei Zhou. 2025. Approximately EFX and fPO Allocations for Bivalued Chores. In *IJCAI*. ijcai.org, 3952–3960.
- [47] Richard J. Lipton, Evangelos Markakis, Elchanan Mossel, and Amin Saberi. 2004. On approximately fair allocations of indivisible goods. In *EC*. ACM, 125–131.
- [48] Ryoga Mahara. 2023. Existence of EFX for two additive valuations. *Discret. Appl. Math.* 340 (2023), 115–122.
- [49] Ryoga Mahara. 2026. Existence of Fair and Efficient Allocation of Indivisible Chores. In *SODA*. SIAM.
- [50] Benjamin Plaut and Tim Roughgarden. 2020. Almost Envy-Freeness with General Valuations. *SIAM J. Discret. Math.* 34, 2 (2020), 1039–1068.
- [51] Ariel D. Procaccia. 2020. An answer to fair division’s most enigmatic question: technical perspective. *Commun. ACM* 63, 4 (2020), 118.
- [52] Barna Saha and Aravind Srinivasan. 2018. A new approximation technique for resource-allocation problems. *Random Struct. Algorithms* 52, 4 (2018), 680–715.
- [53] Ankang Sun and Bo Chen. 2025. Randomized strategyproof mechanisms with best of both worlds fairness and efficiency. *Eur. J. Oper. Res.* 324, 3 (2025), 941–952.
- [54] Ankang Sun, Bo Chen, and Xuan Vinh Doan. 2023. Fairness criteria for allocating indivisible chores: connections and efficiencies. *Auton. Agents Multi Agent Syst.* 37, 2 (2023), 39.
- [55] Biaoshuai Tao, Xiaowei Wu, Ziqi Yu, and Shengwei Zhou. 2025. On the existence of EFX (and Pareto-optimal) allocations for binary chores. *Theor. Comput. Sci.* 1042 (2025), 115248.
- [56] Shengwei Zhou and Xiaowei Wu. 2024. Approximately EFX allocations for indivisible chores. *Artif. Intell.* 326 (2024), 104037. doi:10.1016/J.ARTINT.2023.104037

Algorithm 3: Computation of EF1, MMS, and PO Allocations for Restricted Instances**Input:** Restricted instance $I = (N, M, c, \{c_i\}_{i \in N})$

```

1 Initialize  $B_i \leftarrow \emptyset$  for all  $i \in [n]$ ;
  // Phase 1: Allocation of items in  $M^+$ 
2 Compute an MMS partition  $\mathbf{B} = (B_1, \dots, B_n)$  of  $M^+$  with
  respect to  $c$ ;
  // Modify the partition to ensure EF1
3 while  $\max_{B \in \mathbf{B}} \bar{c}(B) > \min_{B \in \mathbf{B}} c(B)$  do
4    $B_b \leftarrow \arg \max_{B \in \mathbf{B}} \bar{c}(B)$ ;
5    $B_l \leftarrow \arg \min_{B \in \mathbf{B}} c(B)$ ;
6    $e \leftarrow \arg \max_{e' \in B_b} c(e')$ ;
7    $B_l \leftarrow B_l + e$ ;
8    $B_b \leftarrow B_b - e$ ;
  // Phase 2: Allocation of items in  $M^0$ 
9  $R \leftarrow M^0$ ;
10 for  $i \in N$  do
11    $S_i \leftarrow M_i^0 \cap R$ ;
12   Pick an arbitrary bundle  $B \in \mathbf{B}$ ;
13    $B \leftarrow B \cup S_i$ ;
14    $X_i \leftarrow B$ ;
15    $R \leftarrow R \setminus S_i$ ;
16    $\mathbf{B} \leftarrow \mathbf{B} \setminus \{B\}$ ;

```

Output: $\mathbf{X}$ that satisfies EF1, MMS, and PO simultaneously**A Non-existence of EFX allocations for ∞ -restricted instance**

In this section, we consider an alternative and natural extension of the restricted setting for chores, which we refer to as the ∞ -restricted instance.

DEFINITION A.1. A set $\{c_1, c_2, \dots, c_n\}$ of cost functions is ∞ -restricted, if for every $1 \leq i \leq n$, for every item $e \in M$, we have $c_i(e) \in \{\infty, c(e)\}$, where $c(e) \geq 0$. Specifically, if $c_i(e) = c(e)$, we say that item e is relevant to agent i .

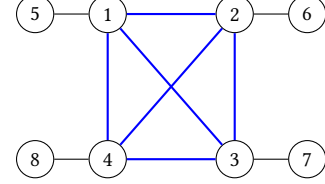
In this model, items cannot be allocated to agents who view them as having infinite cost. Moreover, we assume that for every item $e \in M$, there exists at least one agent i such that $c_i(e) = c(e)$, ensuring that all items are relevant to at least one agent.

Under these assumptions, we study the existence of EFX allocations subject to the restriction that items with infinite cost cannot be assigned to the corresponding agents.

We consider a special case in this setting, where each item e is considered relevant (with cost $c(e)$) by exactly two agents. Such an instance can be represented as an undirected graph, where each vertex corresponds to an agent and each edge corresponds to an item. To compute an EFX allocation, we need to find an EFX orientation on the graph, i.e., for each edge, we determine which of its two incident agents receives the corresponding item, such that the resulting allocation is EFX.

We provide an example (see Example A.2 and Figure 2 for details) that illustrates the nonexistence of EFX allocations in this setting.

EXAMPLE A.2. Consider an ∞ -restricted instance with eight agents and ten items; the structure of agent-item relevance is illustrated in Figure 2. Blue thick edges represent items with a large cost, e.g., $c(e) = 1000$, while black thin edges represent items with a small cost, e.g., $c(e) = 1$. Focusing on agents 1 through 4, note that there are six edges between them. By the pigeonhole principle, at least one of these agents must receive more than one item. Without loss of generality, suppose agent 1 receives more than one item. Then, regardless of how we allocate the item shared between agents 1 and 5, agent 1 will not be EFX towards agent 5, due to receiving multiple large-cost items.

**Figure 2: Illustration of Example A.2.****B EF1 + MMS + PO Allocations**

To prove Theorem 3.8, we present Algorithm 3 and introduce several key observations. For convenience, for agent i and bundle B , we further define

$$\bar{c}_i(B) = \min_{e \in B} \{c_i(B - e)\}$$

as the bundle's cost after removing the item with the largest cost.

We refer to one iteration of the while-loop as a round, and index the rounds by $t = 1, 2, \dots$. We use B_b^t and B_l^t to denote the bundles that we identify at the beginning of round t . We refer to B_b^t and B_l^t as the big bundle and the least bundle in round t , respectively⁹.

OBSERVATION B.1. Let the while-loop on Line 3 run for a total of t rounds. Then for all $1 \leq t' \leq t - 1$, we have $\bar{c}(B_b^{t'}) \geq \bar{c}(B_b^{t'+1})$ and $c(B_l^{t'}) \leq c(B_l^{t'+1})$.

PROOF. In round t' , we select bundles $B_b^{t'}$ and $B_l^{t'}$, and transfer the maximum-cost item e (w.r.t. c) from $B_b^{t'}$ to $B_l^{t'}$. As a result, the cost of $B_b^{t'}$ does not increase in this round. Moreover, since we have $c(B_l^{t'} \cup \{e\}) < c(B_b^{t'})$ by the while-loop condition, the updated cost of $B_l^{t'}$ does not exceed the previous cost of $B_b^{t'}$. The items of all other bundles remain unchanged in this round. Therefore, the value of $\max_{i \in [n]} \{\bar{c}(B_i^{t'})\}$ does not increase in this round, which implies that $\bar{c}(B_b^{t'}) \geq \bar{c}(B_b^{t'+1})$. On the other hand, the updated cost of $B_b^{t'}$ does not lower than the previous cost of $B_l^{t'}$, which implies that the minimum cost (w.r.t. c) will not decrease in this round, leading to the fact that $c(B_l^{t'}) \leq c(B_l^{t'+1})$. $\square$

OBSERVATION B.2. If a bundle B_b^t is selected as the big bundle for the last time in round t , then it can never be selected as the least bundle in any subsequent round.

⁹When selecting the big bundle, we break ties by choosing the bundle with the smallest index. Therefore, we refer to it as "the" big bundle, with a similar rule applied to "the" least bundle.

PROOF. Suppose for contradiction that B_b^t is selected as the least bundle in some later round $t' > t$, and let t' be the smallest such round. Assume that in round t' , the bundle $B_b^{t'}$ is selected as the big bundle, and $B_l^{t'} = B_b^t$ is selected as the least bundle. Since round t is the last round that B_b^t is selected as the big bundle, we have $c(B_l^{t'}) = \bar{c}(B_b^t)$, i.e., items in bundle B_b^t remain unchanged from round t to $t' - 1$. Then in round t' , we have:

$$\bar{c}(B_b^{t'}) > c(B_l^{t'}) = \bar{c}(B_b^t).$$

However, by Observation B.1 and the fact that $t < t'$, we know that

$$\bar{c}(B_b^t) \geq \bar{c}(B_b^{t'}),$$

which leads to a contradiction.

Hence, a bundle that is selected as the big bundle for the last time in some round can never be selected as the least bundle in any subsequent round. $\square$

LEMMA B.3. *The while-loop on Line 3 runs at most nm rounds.*

PROOF. In each round, a bundle is selected as the big bundle, and exactly one item is removed from it. Hence, the size of the bundle decreases by exactly one each time it is selected as the big bundle. By Observation B.2, once a bundle is selected as the big bundle, it can never be selected as the least bundle in any subsequent round, which implies that any bundle can be selected as the big bundle at most m times. Since there are n bundles, the while-loop on Line 3 terminates after at most nm rounds.

We are now ready to prove Theorem 3.8.

By conducting an analysis similar to the one in Lemma 3.4, we can show that at the end of Phase 1, every bundle is MMS-feasible. Additionally, as established in Lemma B.3, the while-loop will eventually terminate. The condition of the while-loop ensures that every bundle also satisfies EF1.

In Phase 2, each agent selects a bundle and adds items that impose zero cost for them, which guarantees that both MMS and EF1 are

preserved. Furthermore, since the final allocation minimizes the social cost, it guarantees Pareto Optimality. Therefore, the final allocation computed by Algorithm 3 satisfies EF1, MMS, and PO simultaneously.

Finally, if we use a PTAS to compute the $(1 - \epsilon)$ -MMS partition in Phase 1 instead, we can guarantee that the overall algorithm runs in polynomial time, thereby proving Corollary 3.9. $\square$

C Cost of Fairness

We remark that the properties in Lemma 4.3 still hold. To upper bound the cost of EFX, we provide an alternative upper bound for the total cost of agents in N_2 .

LEMMA C.1. $\sum_{i \in N_2} c_i(X_i) \leq \frac{1}{2} \cdot \max_{i \in N} c_i(M)$.

PROOF. Note that each agent in N_2 receives her favorite bundle. For any agent $i \in N_2$, we have

$$c_i(X_i) \leq \frac{1}{n} \cdot c_i(M) \leq \frac{1}{n} \cdot \max_{j \in N} c_j(M).$$

Recall that following Lemma 4.2 we have $|N_0| \geq |N_2|$, which implies that $|N_2| \leq n/2$. Hence we have

$$\sum_{i \in N_2} c_i(X_i) \leq \frac{1}{2} \cdot \max_{i \in N} c_i(M). \quad \square$$

Consequently, the cost of EFX can be upper bounded by

$$\begin{aligned} sc(\mathbf{X}) - OPT(I) &= sc(\mathbf{X}) - c(M^+) \\ &= \sum_{i \in N_0 \cup N_1} c_i(X_i) + \sum_{i \in N_2} c_i(X_i) - c(M^+) \\ &\leq \frac{1}{2} \cdot \max_{i \in N} c_i(M). \end{aligned}$$

This implies that

$$\frac{sc(\mathbf{X}) - OPT(I)}{\max_{i \in N} c_i(M)} \leq \frac{1}{2}.$$